

Reviewer Pack — Minimal Rank of Geometric Quantization over Tseitin Resoluti...

Entry 6f539896706d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 22:38:17 UTC

Minimal Rank of Geometric Quantization over Tseitin Resolution Trees

Entry ID: 6f539896706d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 22:38:17 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization **Field B** (complexity object): Tseitin Resolution Trees

Statement:

[‘For any given Tseitin formula, the minimal rank of its associated quantization bundle is at least exponential in the depth of its resolution tree.’, ‘This implies that a Tseitin formula requiring Resolution refutation of length λ has a quantization bundle with rank at least $2^{(\Omega(\lambda))}$.’, ‘There exists a polynomial-time computable mapping from any Tseitin formula to its corresponding geometric quantization bundle.’]

Rationale (proposer’s reasoning):

[‘Geometric quantization has been successfully applied in algebraic geometry and symplectic topology, providing new insights into the structure of classical mechanical systems.’, ‘By applying this framework to Tseitin resolution trees, we may uncover hidden structures in the complexity of satisfiability problems.’, ‘The conjecture suggests a deep connection between geometric quantization and computational complexity theory, potentially leading to new algorithmic advancements.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 39ed62473acffc83

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Tseitin formulas up to depth $n=40$, the minimal rank of the quantization bundle is at least exponential in the depth with a ratio of at least $2^{(0.5n)}$ and no seed produces a rank less than $2^{(0.4n)}$. The conjecture is falsified if for any seed or depth, the rank is found to be less than $2^{(0.4n)}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric quantization" AND "Tseitin resolution trees" - "minimal rank" IN geometer quantization AND Tseitin resolution tree" - "polynomial-time mapping" FROM Tseitin formula TO geometric quantization bundle

Top relevant hits considered: - [http://arxiv.org/abs/2605.11396v1] MuonQ: Enhancing Low-Bit Muon Quantization via Directional Fidelity Optimization - [http://arxiv.org/abs/1710.07343v1] Fractional Minimal Rank - [http://arxiv.org/abs/2205.13954v2] Geometer: Graph Few-Shot Class-Incremental Learning via Prototype Representation - [http://arxiv.org/abs/math-ph/0011018v1] Geometric Quantization on the Super-Disc - [http://arxiv.org/abs/1802.04919v2] New Universal Deformation Formulas for deformation quantization - [http://arxiv.org/abs/2601.23259v1] Geometric Quantization by Paths, Part III: The Metaplectic Anomaly

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(depth):
        if depth == 1:
            return random.choice(['A', 'B'])
        else:
            p = generate_tseitin_formula(random.randint(1, depth-1))
            q = generate_tseitin_formula(random.randint(1, depth-1))
            return f'({p} & {q}) | ({p} -> {random.choice(["A", "B"])}'

    def tseitin_resolution_tree(formula):
        # Simplified version of Tseitin resolution tree generation
        if formula.isalpha():
            return [formula]
        elif formula.startswith('(') and formula.endswith(')'):
            parts = formula[1:-1].split()
            if len(parts) == 3:
                p, op, q = parts
```

```

        if op == '&':
            return tseitin_resolution_tree(p) + tseitin_resolution_tree(q)
        elif op == '|':
            return tseitin_resolution_tree(f'~{p}') + tseitin_resolution_tree(f'~{q}')
    return []

def geometric_quantization_rank(tree):
    # Simplified version of geometric quantization rank calculation
    if not tree:
        return 0
    else:
        return max(geometric_quantization_rank(subtree) for subtree in tree) + 1

depth = random.randint(5, 40)
formula = generate_tseitin_formula(depth)
tree = tseitin_resolution_tree(formula)
rank = geometric_quantization_rank(tree)

metric_value = rank
instances_tested = 1
conjecture_holds = rank >= 2**(0.4 * depth)
counterexample = f'Depth {depth}, Rank {rank}' if not conjecture_holds else ''

return {
    "metric_name": "geometric_quantization_rank",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30*2 + 1, 2)) # Default to first 30 p

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Depth {results[0]['counterexample'].split(',')[0]}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
TRIAL: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
TRIAL: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
TRIAL: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
TRIAL: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
TRIAL: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
TRIAL: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
TRIAL: {'metric_name': 'geometric_quantization_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture': True}
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe171781.py", line 85, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe171781.py", line 85, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results show that for several depths, the minimal rank of the quantization bundle is 0, which contradicts the conjecture that it should be at | next: Investigate why the test produced a rank of 0 for certain depths and whether this is an error in the implementation or indicative of a flaw in the conjecture itself.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10927
2	preregistration	ollama_remote	glm4:latest	0	0	5984
3	novelty	ollama_remote	glm4:latest	0	0	4648
4	novelty	ollama_remote	glm4:latest	0	0	6139
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12629
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35749
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16137
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10579
9	judge	ollama_remote	glm4:latest	0	0	50975

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 153766 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/6f539896706d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6f539896706d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6f539896706d.tar.gz (if generated)